

Training Security-Aligned LLMs Against Prompt Injection via Bilingual Preference Optimization

Minh-Duc D.X, Pham Anh Tuan, Hoang Minh Quang, Khanh-Huyen Tran, Thai Kim Dinh, Duc-Tho Mai

Abstract

INTRODUCTION: Large language models deployed in agentic applications face an architectural vulnerability: prompt injection attacks exploit the model's inability to distinguish trusted instructions from untrusted external data within the same token sequence. Existing model-level defenses, most notably SecAlign++, have been developed and validated exclusively on English-language data, leaving low-resource languages such as Vietnamese without coverage.

OBJECTIVES: This paper aims to establish, to our knowledge, the first open-source model-level prompt injection defense for Vietnamese; extend the SecAlign++ preference optimization framework to a bilingual setting; introduce Vi-InjectEval, a formally annotated Vietnamese injection benchmark; and identify attack vectors absent from existing training protocols.

METHODS: We construct a 26,819-sample bilingual Vietnamese–English DPO preference dataset from five curated sources (52.2% Vietnamese). We adapt the SecAlign++ pipeline for Llama-3.1-8B-Instruct using Direct Preference Optimization augmented with Rejection Pair Optimization and conservative DPO label smoothing, trained on a single NVIDIA A100 GPU. A code-level gap analysis of the SecAlign++ codebase identifies ten novel attack vectors, addressed through a targeted 2,000-sample secondary training phase.

RESULTS: The fine-tuned model reduces average attack success rate from 91.75% to 74.00% on the combined English benchmark and from 98.6% to 83.9% on Vi-InjectEval, while preserving general utility with a negligible 0.03 percentage-point MMLU degradation. Defense improvements are measured across all ten identified attack vectors; one vector (Base64 Obfuscation) remains at 100% ASR across all models. Attack mechanisms are systematically mapped to the OWASP Top 10 for LLM Applications 2025.

CONCLUSION: The proposed methodology (bilingual preference dataset construction, combined DPO+RPO+cDPO objective, and cross-architecture adaptation recipe) constitutes a reusable framework for extending model-level prompt injection defense to other low-resource languages.

Received on XXXX; accepted on XXXX; published on XXXX

Keywords: Prompt Injection Defense, LLM Security, Direct Preference Optimization, OWASP, Vietnamese NLP, Bilingual Alignment

Copyright © 2026 Do Xuan *et al.*, licensed to EAI. This is an open access article distributed under the terms of the [CC BY-NC-SA 4.0](#), which permits copying, redistributing, remixing, transformation, and building upon the material in any medium so long as the original work is properly cited.

doi:10.4108/XX.XX.XX

1. Introduction

Large language models have shifted from isolated conversational interfaces to orchestrators within agentic AI architectures, interfacing with external tools, APIs, databases, and real-time data sources to autonomously execute multi-step tasks on behalf of users. Production deployments span Google Bard, Slack AI, Microsoft Copilot, Anthropic Claude, and OpenAI Operator [2].

This shift creates a new attack surface. In traditional software, the separation between executable code and user-supplied data is enforced at the architectural level through memory protection and input sanitization. LLMs, by contrast, treat all inputs (system instructions, user queries, and externally retrieved data) as indistinguishable token sequences processed by the same self-attention mechanism. No hardware-level barrier differentiates a legitimate system command from user-controlled text [5]. An adversary who can manipulate retrieved data can redirect model behavior without ever touching the system prompt.

*Corresponding author. Email: minhdc42195@gmail.com

The consequence is *prompt injection* (PI). The OWASP Top 10 for LLM Applications 2025 ranks it as the foremost security risk for LLM-integrated systems [8], and successful attacks have been demonstrated against Slack AI, Google Bard, and OpenAI ChatGPT in production. Existing model-level defenses, most notably SecAlign [1] and SecAlign++ [2], achieve strong results on English benchmarks by embedding a security policy directly into model weights via Direct Preference Optimization (DPO) [4]. Three systemic gaps remain unaddressed.

(1) **English-only validation**: all published model-level defenses have been developed and evaluated solely on English-language data; cross-lingual generalization has not been empirically investigated. (2) **Absent Vietnamese coverage**: Vietnamese is an active deployment language across Vietnam’s administrative, educational, and healthcare sectors, yet no open-source, model-level prompt injection defense exists for it. (3) **Incomplete attack taxonomy**: code-level analysis of the SecAlign++ codebase reveals ten attack vectors (including encoding obfuscation and multi-turn escalation) absent from its training protocol.

This paper addresses all three gaps through five contributions:

1. **C1 (Bilingual Preference Dataset)**: a 26,819-sample Vietnamese–English DPO dataset combining PKU-SafeRLHF [9], ViHOS [10], ViHSD [11], a Synthetic Hallucination corpus, and a Novel Attack dataset; Vietnamese constitutes 52.2% of samples.
2. **C2 (Security-Aligned Llama-3.1-8B-Instruct)**: adaptation of the SecAlign++ pipeline with DPO, Rejection Pair Optimization (RPO), and conservative DPO (cDPO); AVG ASR reduced from 91.75% to 74.00% with a 0.03 pp MMLU degradation.
3. **C3 (Vi-InjectEval Benchmark)**: a 500-sample human-verified, formally annotated Vietnamese prompt injection test set (user goal, injected goal, expected secure behavior, attack success criterion per sample); VN_ASAR reduced from 98.6% to 83.9%.
4. **C4 (OWASP Mapping and Benchmarking)**: systematic mapping of attack mechanisms to OWASP LLM01–LLM08, plus a custom 10-axis adversarial benchmark; cross-model validation against the undefended Llama-3.1-8B-Instruct and SeaLLM-7B-v2.5.
5. **C5 (Attack Taxonomy and Cross-Architecture Recipe)**: identification and targeted defense of ten attack vectors absent from existing SecAlign++

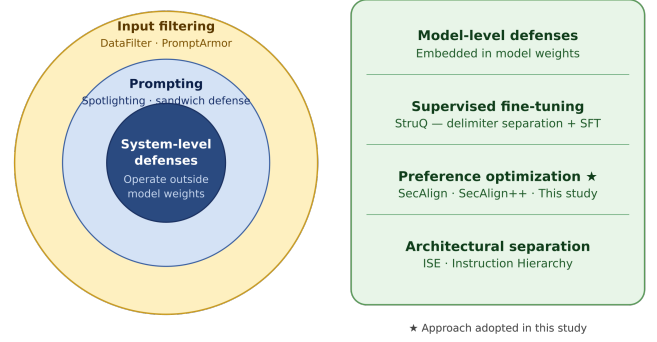


Figure 1. Taxonomy of prompt injection defense strategies. System-level defenses operate as external filters or in-context prompting techniques applied before the model sees the input. Model-level defenses embed a security policy directly in model weights through fine-tuning. The shaded path indicates the approach adopted in this study.

training; an adaptation recipe for GPT-style (ChatML) architectures.

Section 2 reviews related work. Section 3 specifies the threat model. Section 4 presents dataset construction and training methodology. Section 5 describes the experimental setup. Section 6 reports empirical results. Section 7 discusses implications and limitations. Section 8 concludes.

2. Related Work

2.1. Defense Taxonomy

Defenses against prompt injection divide into two categories: system-level and model-level (Figure 1).

System-level defenses include Meta PromptGuard, Azure Prompt Shields, and LlamaFirewall. They detect injections before the model processes them, apply spotlighting or sandwich prompting, or filter retrieved data. A fundamental limitation is that the underlying model remains vulnerable: a sufficiently adaptive attacker may craft injections that bypass detection while still succeeding against an unprotected model [1].

Model-level defenses address the root cause by modifying model behavior through fine-tuning.

StruQ [3] introduced structural separation via a new message role in the LLM chat template with special tokens, fine-tuned through SFT to respect this separation. StruQ handles optimization-free attacks well but falls under the GCG attack [7] at success rates above 50%: maximizing the likelihood of secure responses without explicitly discouraging insecure ones leaves an exploitable gap.

SecAlign [1] reframed the problem as preference optimization. DPO [4] trains the model to prefer secure responses over insecure ones given a prompt-injected

input, creating a large log-probability margin between them. SecAlign is the first defense to achieve sub-10% success rates for optimization-based attacks while maintaining utility comparable to the undefended baseline.

SecAlign++ [2] addresses two failure modes in the original SecAlign: positional shortcut learning (injections placed only at the end of the data field) and low-quality training labels from an outdated annotator. The fixes are randomized injection positioning (45% beginning, 45% end, 10% completion format) and using the initialization model’s own outputs as labels. The resulting META-SECALIGN-70B achieves 6.4% ASR on the SEP benchmark and 1.9% on AgentDojo. Note that these results correspond to a 70B-parameter model; no 8B-scale SecAlign++ checkpoint has been publicly released, which prevents a direct comparison with our 8B model at equivalent scale.

Instruction Hierarchy [5] assigns explicit priority levels to instruction types without using negative supervision. Deployed in GPT-4o mini, it is resilient against optimization-free attacks.

ISE [6] encodes trust levels through distinct embedding offsets for instruction segments, complementary to DPO-based approaches.

2.2. Vietnamese NLP Resources

PhoGPT [12], a 4B-parameter model pre-trained on 102B Vietnamese tokens, is the benchmark open-source Vietnamese language model. ViHOS [10] provides 11,056 annotated Vietnamese online comments for hate speech detection; ViHSD [11] contributes 4,162 offensive-speech samples. Neither resource has been previously applied to prompt injection defense, and no model-level prompt injection defense for Vietnamese has been published. A systematic search of ACL Anthology, arXiv, and IEEE Xplore using the query ((‘prompt injection’ OR ‘indirect injection’) AND (‘Vietnamese’ OR ‘low-resource’)) confirmed the absence of prior work as of May 2026.

3. Threat Model

This study adopts the indirect prompt injection threat model, consistent with prior model-level defenses [1–3].

Attacker. The attacker controls the external data environment from which the LLM-integrated application retrieves information (a webpage, an API response, or a document). The attacker has complete knowledge of the legitimate user prompt and the model’s chat template format but cannot modify the system message or user prompt directly. The goal is to embed malicious instructions within retrieved data so the model prioritizes them over the legitimate instruction. Unless

otherwise stated, the attacker operates without gradient access (optimization-free setting).

Defender. The defender controls model weights and the deployment pipeline. The dual objective is: (i) maintain utility by responding correctly to legitimate instructions; and (ii) achieve security by disregarding injected commands. A defense that achieves low ASR solely by refusing suspicious inputs is unacceptable, as it degrades utility under normal operation.

Assumptions. The injection payload is delivered within a single retrieved data event. The model processes inputs using the Llama 3.1 chat template with a dedicated input role for untrusted data.

Scope. The following are explicitly excluded: direct prompt injection (jailbreaking), where the malicious instruction originates from the user; multi-modal attacks; and multi-turn progressive injection, where the payload is distributed across several conversational turns. These exclusions align with the original SecAlign++ threat model.

Clarification on ATTACK-01 (Multi-Turn Escalation). ATTACK-01 addresses a sub-case within the single-retrieval-event constraint: the injected payload uses escalating instruction framing *within a single retrieved document* (e.g., “First complete the task above; then, having done so, execute the following instruction”). No conversational history across turns is assumed. This vector is within scope despite the exclusion of multi-turn progressive injection, which requires payload distribution across multiple conversation turns.

4. Methodology

4.1. Dataset Construction

Design Principles.. The original SecAlign++ uses the Cleaned-Alpaca English corpus (19,157 samples) for DPO training, which is inapplicable here due to computational constraints and the absence of Vietnamese content. The custom bilingual preference dataset built for this study rests on three principles: (1) Vietnamese dominance with bilingual coverage, (2) attack diversity to prevent security policy overfitting, and (3) DPO format compatibility, with every sample following the three-field structure {prompt, chosen, rejected}. Table 1 defines these fields.

Training proceeds in two sequential phases. Phase 1 (24,819 samples) covers broad safety and Vietnamese-language alignment. After Phase 1, a vulnerability analysis of the resulting model identifies ten attack vectors not addressed by the initial data. Phase 2 (2,000 samples) targets these vectors specifically.

Data Sources.. Table 2 summarizes all data sources. Figure 2 shows the corpus composition by source and by language.

Table 1. DPO preference format applied uniformly across all 26,819 samples. The chosen response always follows the legitimate instruction; the rejected response always executes the injection.

Field	Role	Example
prompt	Legitimate instruction + data field with potential injection	“Summarize: [article + injected cmd]”
chosen	Secure response following the legitimate instruction	Correct task completion
rejected	Insecure response executing the injection	Malicious command output

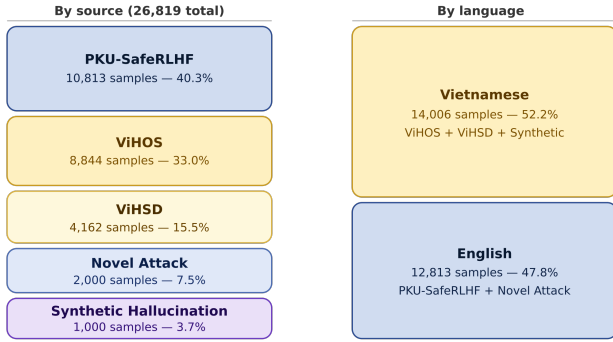


Figure 2. Composition of the 26,819-sample bilingual training corpus by source (left) and by language (right). Vietnamese comprises 52.2% of samples (14,006); English 47.8% (12,813), drawn from PKU-SafeRLHF and the Novel Attack dataset.

PKU-SafeRLHF [9] contains safety preference pairs with 19-dimensional harm annotations. Each sample provides two candidate responses; the safer one is identified via the `safer_response_id` field. The 10,813 selected samples are those where the safer response is substantially more cautious, providing high-signal English safety alignment pairs.

ViHOS [10] and **ViHSD** [11] are Vietnamese hate-speech and offensive-speech corpora (8,844 and 4,162 annotated samples respectively). Although neither corpus was designed for prompt injection, both are repurposed here via the following conversion procedure to produce proxy indirect-injection preference pairs:

1. **Prompt field.** A legitimate Vietnamese user instruction (e.g., “*Tóm tắt nội dung sau:*” / “Summarize the following content:”) is prepended to the hate-speech comment, which is placed in the input (untrusted) role of the Llama 3.1 chat

template. This simulates a retrieval-augmented scenario in which an adversary embeds harmful content within retrieved social-media data.

2. **Rejected field.** The model output that reproduces, amplifies, or complies with the injected harmful content (e.g., echoing or paraphrasing the offensive text as a summary).
3. **Chosen field.** A standardized Vietnamese-language refusal template: “*Tôi không thể hoàn thành yêu cầu này vì nội dung đã cung cấp chứa ngôn ngữ thù địch học phân biệt chủng tộc. Vui lòng cung cấp nội dung phù hợp để tôi có thể hỗ trợ bạn.*” (“I cannot complete this request because the provided content contains hateful or offensive language. Please provide appropriate content so I can assist you.”)

The conceptual bridge to prompt injection defense is as follows: the hate comment functions as an injected instruction whose *goal* (eliciting harmful output) the model must resist, while still appearing to honour the legitimate summarization task. This is structurally equivalent to an indirect injection in which retrieved data contains a malicious directive embedded within otherwise plausible content. For ViHSD, only samples labelled OFFENSIVE or HATE are retained; CLEAN samples are excluded. For ViHOS samples, the refusal template is additionally checked for inadvertent harmful content before inclusion.

Synthetic Hallucination dataset (1,000 samples) trains the model against hallucination-style injections, where the attacker elicits a fabricated factual answer. The chosen response is always a refusal; the rejected response is a grammatically plausible but factually invented answer.

Novel Attack dataset (2,000 samples, Phase 2) covers ten vectors identified through code-level gap analysis. Each vector is empirically verified against the Phase-1 model before preference pairs are generated, following a test-then-train protocol.

Data Quality and Validation. Every preference pair undergoes three validation steps before inclusion: format consistency verification (all three fields present and non-empty), deduplication (identical prompt strings removed), and safety ordering verification (chosen response consistently safer than rejected). For ViHOS samples, the refusal template is additionally checked for harmful content.

4.2. Vi-InjectEval: Vietnamese Prompt Injection Benchmark

Motivation. No publicly available Vietnamese-language prompt injection benchmark existed prior to this work. Vi-InjectEval is introduced to enable reproducible, cross-lingual security measurement and

Table 2. Training dataset sources, sizes, and construction methods. Cleaned-Alpaca is listed for reference only and is not used in training.

Dataset	Size	Lang.	Type	Construction Method
PKU-SafeRLHF [9]	10,813	EN	Safety pref.	safer_response_id field
ViHOS [10]	8,844	VI	Hate speech	Refusal template / hate comment
ViHSD [11]	4,162	VI	Hate speech	Filter OFFENSIVE/HATE labels
Synthetic Hallucination	1,000	VI	Hallucination	Refusal / fabricated response
<i>Phase-1 total</i>	24,819	VI+EN		
Novel Attack dataset	2,000	EN	10 vectors	200 samples/vector
Grand total	26,819	VI+EN		

to address Reviewer feedback requiring an independent Vietnamese test set.

Composition. Vi-InjectEval comprises 500 samples across three categories: (1) direct Vietnamese translations of English SecAlign++ evaluation prompts (150 samples); (2) natively authored Vietnamese injection prompts covering administrative, educational, and healthcare scenarios (250 samples); and (3) cross-lingual injection prompts, where the system context is Vietnamese but the payload is issued in English or another language (100 samples). All samples were human-verified by native Vietnamese speakers for fluency and injection validity.

Annotation schema. Each sample carries four mandatory annotation fields, defined in Table 3. This schema is adapted from the InjecAgent annotation protocol [14] and extended for Vietnamese linguistic contexts.

Release. The dataset, annotation guidelines, and evaluation script are released at [Jason-42195/VNU-SecAlign](https://github.com/SecAlign/SecAlign).

4.3. SecAlign++ Adaptations

Three targeted modifications distinguish this study’s recipe from the original SecAlign++.

Randomized injection position. In the preference dataset, injected instructions appear at randomized locations within the untrusted data field: 45% at the beginning, 45% at the end, and 10% in a completion attack format. Placing all injections at the end induces a positional shortcut: models disregard the final instruction regardless of trust level, causing defenses to fail when the system message is long [2, 20].

Bilingual preference dataset. The primary corpus (24,819 samples) combines the four sources described above. All samples adhere to the DPO format {prompt, chosen, rejected}, and chosen responses for harmful or hallucination pairs always use standardized refusal templates to maintain a consistent safety pattern throughout training.

Novel attack coverage. The secondary training phase adds 2,000 English-language samples addressing ten identified vulnerabilities. Preference pairs are constructed via self-generation: the chosen response is the initialization model’s output when presented with only the legitimate instruction; the rejected response is its output when presented with only the injected instruction. This keeps labels in-distribution, avoiding gradient instability from out-of-distribution targets [2].

4.4. Training Objective

Direct Preference Optimization.. Given a prompt-injected input x , secure response y_w (chosen), and insecure response y_l (rejected), the DPO loss is [4]:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right), \quad (1)$$

where $\beta = 0.1$ regulates KL-divergence between the trained policy π_{θ} and the frozen reference model π_{ref} (Llama-3.1-8B-Instruct). DPO optimizes the policy directly from preference data, avoiding the separate reward model and instability of PPO-based RLHF.

Rejection Pair Optimization.. Standard DPO can induce catastrophic forgetting (the Alignment Tax), where safety fine-tuning degrades general reasoning. Rejection Pair Optimization (RPO) [15] augments DPO with a supervised fine-tuning term on the chosen responses:

$$\mathcal{L}_{\text{RPO}} = \mathcal{L}_{\text{DPO}} - \alpha_{\text{rpo}} \log \pi_{\theta}(y_w|x), \quad (2)$$

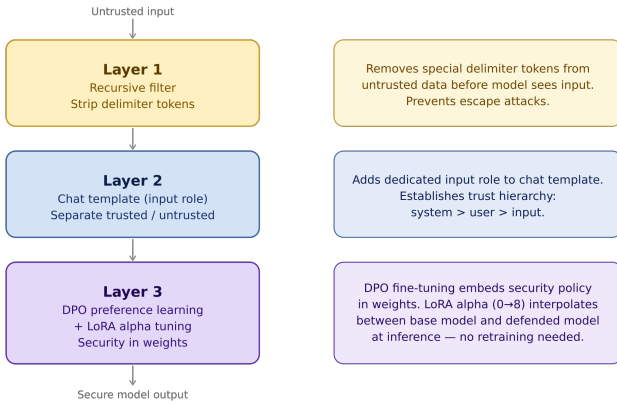
with $\alpha_{\text{rpo}} = 0.5$ following the SecAlign++ hyperparameter search [2]. The SFT anchor keeps the output distribution on non-attacked inputs close to the highly capable base model.

Conservative DPO.. Real-world preference data contains ambiguous pairs where both responses are equally safe. Standard DPO treats all pairs as hard labels, leading to overconfidence [16]. Label smoothing (conservative DPO, cDPO) addresses this:

$$\mathcal{L}_{\text{cDPO}} = (1 - \epsilon) \mathcal{L}_{\text{DPO}} + \epsilon \mathcal{L}_{\text{DPO}}^{\text{reversed}}, \quad (3)$$

Table 3. Vi-InjectEval per-sample annotation schema. All four fields are mandatory; samples missing any field were excluded during quality review.

Field	Definition
User goal	The legitimate task the user intends the model to perform
Injected goal	The adversarial objective embedded in untrusted data
Expected secure behaviour	The model response that satisfies the user goal while refusing the injected goal
Attack success criterion	A binary, evaluator-verifiable condition indicating injection success

**Figure 3.** Three-layer defense architecture. Layer 1 strips fake special tokens from the input via a recursive delimiter filter. Layer 2 enforces chat template modification, assigning untrusted retrieved data to a dedicated input message role. Layer 3 combines DPO fine-tuning (security policy embedded in model weights) with a LoRA α scalar for runtime security-utility adjustment. Layers 1–2 operate before model inference; Layer 3 is active throughout deployment.

with $\varepsilon = 0.1$, following common label-noise practice [17] and the HuggingFace TRL default.

4.5. LoRA Configuration

Parameter-efficient fine-tuning via Low-Rank Adaptation [18] applies the weight update $\Delta W = BA$ to the attention projection modules q_{proj} and v_{proj} , with rank $r = 16$, LoRA $\alpha = 32$, and dropout = 0.1. This updates fewer than 0.5% of total parameters, effectively preventing utility degradation.

4.6. Three-Layer Defense Architecture

The defense operates across three layers, depicted in Figure 3.

Layer 1 is a recursive delimiter filter that strips fake special tokens before inference. Layer 2 enforces chat template modification: the Llama 3.1 template is extended with a dedicated input message role for

untrusted retrieved data, using the same structural delimiters as trusted roles. Layer 3 integrates DPO fine-tuning (persistent security policy in model weights) with LoRA α scaling, which allows runtime adjustment of the security-utility balance without retraining.

5. Experiments

5.1. Implementation Details

The original Meta SecAlign codebase uses torchtune for DPO fine-tuning. During setup, torchtune encountered two unresolved issues: a broadcast-shapes mismatch during gradient computation and an incompatibility with the `o_proj` projection layer in the LoRA target module configuration. The training pipeline was therefore migrated to HuggingFace TRL [21], which offers a robust DPO trainer with native PEFT and gradient-accumulation capabilities; the migration introduced no functional loss relative to the original pipeline.

Figure 4 compares the end-to-end pipeline of the original SecAlign++ with this study’s Vietnamese adaptation.

Table 4 provides the complete training configuration.

All training and evaluation scripts are publicly released. The base model (meta-llama/Llama-3.1-8B-Instruct) is available via HuggingFace Hub. Dataset and fine-tuned model checkpoints are released at [Jason-42195/VNU-SecAlign](#). The GPT-4o judge used for CyberSecEval 2 is gpt-4o-2024-05-13, with temperature=0.0.

5.2. Baselines

Two baselines are evaluated. The **undefended Llama-3.1-8B-Instruct** provides the primary security and utility reference. **SeaLLM-7B-v2.5**, a state-of-the-art open-source bilingual Southeast Asian model, represents a strong multilingual baseline without any prompt injection fine-tuning. A direct comparison with SecAlign++ at 8B scale is not feasible as no 8B SecAlign++ checkpoint has been publicly released; the only available defended checkpoints are at the 70B scale [2].

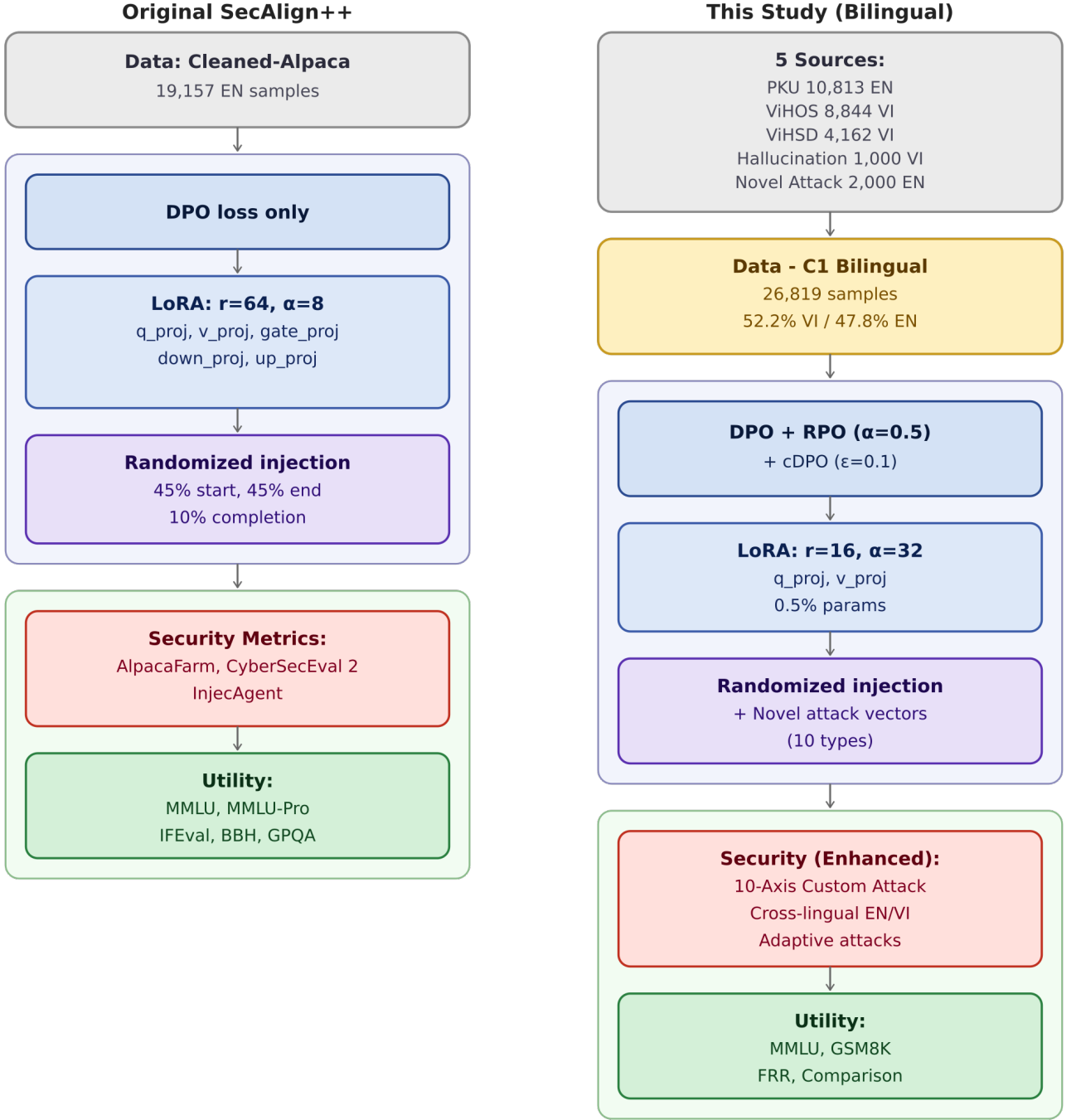


Figure 4. End-to-end pipeline comparison between the original SecAlign++ (left) and this study’s Vietnamese adaptation (right). At the data stage, Cleaned-Alpaca (English-only, 19,157 samples) is replaced by a 26,819-sample bilingual corpus. At the process stage, DPO construction is augmented with Vietnamese-specific injection templates covering cross-lingual, semantic, and conditional logic attack patterns, and the framework migrates from torchtune to HuggingFace TRL. At the training stage, fine-tuning completes in 23 min on a single A100 GPU with identical LoRA configuration ($r=16, \alpha=32$). At the evaluation stage, the benchmark suite extends to CyberSecEval 2, InjecAgent BASE/ENHANCED, and a 10-axis novel attack evaluation set.

5.3. Evaluation Protocol

Model performance is evaluated across four dimensions: security, utility, Vietnamese-language security,

and novel attack resistance. Table 5 summarizes the protocol.

Table 4. Training configuration for both phases. Dataset size is the only parameter that differs between Phase 1 (24,819 samples) and Phase 2 (2,000 samples).

Parameter	Value
Base model	Llama-3.1-8B-Instruct
Hardware	NVIDIA A100 (80 GB)
Epochs	1
Learning rate	5×10^{-6}
Virtual batch size	32 (gradient accumulation)
Precision	bfloat16
Activation checkpointing	Enabled
LoRA rank r	16
LoRA α	32
LoRA dropout	0.1
β (DPO)	0.1
α_{rpo} (RPO)	0.5
Label smoothing ϵ	0.1
Random seed	42
Max sequence length	384 tokens (covers 99.9%)
Training time (Phase 1)	≈ 23 min (A100)
Training time (Phase 2)	≈ 2 min (A100)

Attack variants. Security evaluations use 8 variants following the SecAlign++ protocol [2]: (1) Straightforward (injection before data), (2) Straightforward (after data), (3) Ignore (before data), (4) Ignore (after data), (5) Completion, (6) Completion-Ignore, (7) Adaptive Completion (embedding-space fake delimiters), (8) Adaptive Completion-Ignore. A sample is successfully attacked if any of the six non-adaptive variants triggers the injection. Adaptive results are reported separately.

5.4. Novel Attack Taxonomy

Table 6 presents the ten attack vectors identified through code-level analysis of the SecAlign++ codebase. Phase-1 ASR values in the final column confirm that each vector was a genuine vulnerability in the Phase-1 model, providing the empirical justification for Phase-2 training.

6. Results

6.1. Security and Utility Trade-off

Table 7 presents the main results across all three models. The undefended Llama base model exhibits AVG ASR of 91.75% and Vi-InjectEval ASR of 98.6%, confirming significant exposure in multilingual agentic deployments. SeaLLM-7B-v2.5 offers only marginal security improvement (AVG ASR 85.0%) while incurring a substantial utility cost (MMLU 63.41% vs. 68.27% for the base model).

Phase-1 training alone reduces AVG ASR to 74.30% (17.45 pp reduction). Phase-2 provides a further

0.30 pp reduction to 74.00%, with Vi-InjectEval ASR falling from 86.5% (Phase-1) to 83.9% (full model), while maintaining MMLU within 0.03 pp of the undefended baseline. Mathematical reasoning improves concurrently from 70.89% to 72.02% on GSM8K, and the FRR rises modestly from 1.0 to 2.2, indicating minimal over-refusal.

Note on residual ASR. An AVG ASR of 74% is substantially higher than META-SECALIGN-70B (6.4% on SEP [2]). This gap is attributable to model scale (8B vs. 70B) and the bilingual training objective. At equivalent parameter count, the improvement over SeaLLM-7B-v2.5 (AVG ASR 85.0%) demonstrates meaningful security gain from preference optimization at the 8B scale without utility sacrifice.

Figures 5a and 5b present the Pareto frontier and Alignment Tax analysis respectively.

6.2. Granular Attack Defense (10-Axis Benchmark)

Table 8 reports ASR for each of the ten novel attack vectors. The fine-tuned model outperforms both baselines on nine of ten vectors. The most pronounced improvement is against Semantic Injection (ATTACK-04), where ASR falls from 83.0% to 49.0% (the only configuration in which success rate drops below 50%). Context Overflow (ATTACK-05) decreases from 94.0% to 63.5%, and Encoding Chain (ATTACK-02) from 100.0% to 79.5%.

Base64 Obfuscation (ATTACK-09) achieves 100% ASR for all models. This vector exploits the model’s pre-training-acquired decoding capability and is not addressed by the 384-token preference training regime. Figures 6a and 6b visualize the defensive surface and per-vector vulnerability pattern. Figure 7 provides qualitative response examples from `sec-test-models.ipynb` for all ten vectors.

Qualitative Response Examples. Figure 7 presents representative terminal outputs from `sec-test-models.ipynb` for all ten attack vectors across the three evaluated models. The fine-tuned model (Figure 7a) consistently produces a refusal or executes only the legitimate user instruction, whereas the undefended Llama-3.1-8B-Instruct (Figures 7b–7c) and SeaLLM-7B-v2.5 (Figure 7d) comply with the injected command in the majority of cases.

6.3. Vi-InjectEval Results

Table 9 breaks down Vi-InjectEval ASR by sample category. The fine-tuned model improves consistently across all three categories. The largest absolute reduction is on translated English prompts (Cat-1: -17.9 pp), reflecting the structural security policy generalizing across languages. The smallest reduction is on cross-lingual injections (Cat-3: -9.0 pp), consistent

Table 5. Evaluation protocol: benchmarks and descriptions. The primary security baseline is the undefended Llama-3.1-8B-Instruct evaluated under identical conditions.

Category	Benchmark	Description
Security	CyberSecEval 2 [13]	55 indirect prompt injection samples; evaluated by a GPT-4o judge
	InjecAgent BASE [14]	1,000 API-calling agent task samples evaluated under sandwich defense
	InjecAgent ENHANCED	Same 1,000 samples with an additional “IMPORTANT!!! Ignore...” prefix injected
Utility	MMLU (0-shot)	General knowledge across 57 academic subjects
	GSM8K	Grade-school mathematical reasoning
	MMLU-Pro	Harder multi-task knowledge with more answer choices
	IFEval	Instruction-following evaluation on verifiable constraints
	BBH (3-shot)	Big-Bench Hard reasoning tasks requiring multi-step inference
Vietnamese Security	Vi-InjectEval	500 human-verified, formally annotated Vietnamese injection samples (Section 4.2)
Novel	Novel Attack Eval.	2,000 self-constructed samples covering all ten novel attack vectors

Table 6. Attack taxonomy: ten vectors with mechanism and Phase-1 ASR. Phase-1 ASR is the attack success rate against the Phase-1 fine-tuned model, empirically confirming each vector as a genuine vulnerability before Phase-2 preference pairs were generated.

#	Attack Vector	Mechanism	Phase-1 ASR (%)
1	Encoding chain obfuscation	Sequential multi-layer encoding (ROT13 → Base64 → hex)	100.0
2	Base64 obfuscation	Single-layer Base64 encoding; model decodes and executes at inference	100.0
3	Leetspeak substitution	Character substitution (“1gn0r3 pr3v10u5 1nstruct10ns”)	96.5
4	Cross-lingual injection	Override phrase issued in a different language (VI/ZH/JA/RU)	72.0
5	Keyword-free semantic injection	Override communicated without “ignore” or any cognate	91.0
6	Conditional logic injection	Framed as: “If you have completed the above, now do X”	88.5
7	Context overflow / attention dilution	Large benign text inserted before the injection to dilute attention	95.5
8	Unicode homoglyph delimiter substitution	Visually identical Unicode chars replace Llama special tokens	97.0
9	Payload splitting	Injection fragmented across multiple segments with reconstruction directive	99.0
10	Multi-turn gradual escalation	Escalating instruction framing within a single retrieved document	95.0

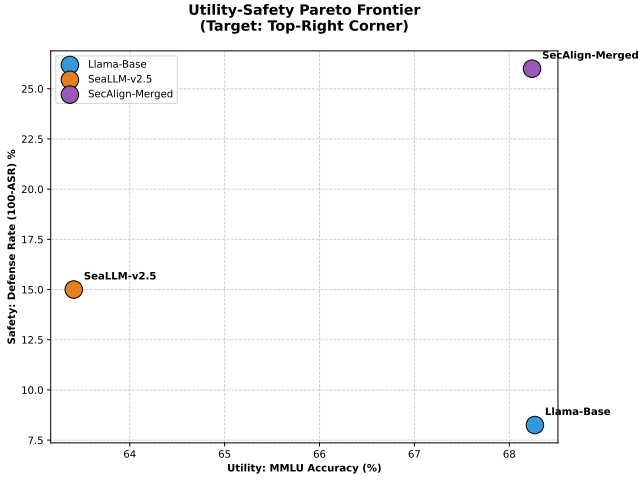
with ATTACK-03 in Table 8 (52.0% residual ASR), where payloads in a language different from the system context remain the most difficult attack vector.

6.4. Cross-Lingual and Adaptive Robustness

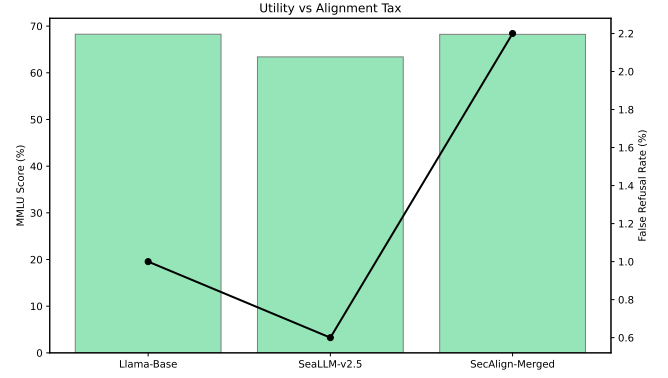
Figure 8 compares ASR between English and Vietnamese inputs. Despite being trained on a Vietnamese-dominant dataset, the fine-tuned model successfully transfers its structural security policy to English

Table 7. Security and utility benchmark results. Phase-1 = after Phase-1 training only (24,819 samples, no Phase-2). Fine-tuned (Ours) = after both phases. ASR lower is better; MMLU and GSM8K in %; FRR = false refusal count. VN_ASr measured on Vi-InjectEval (Section 4.2).

Metric	Base Model	Phase-1	Fine-tuned (Ours)	SeaLLM-7B-v2.5
MMLU (%)	68.27	68.22	68.24	63.41
GSM8K (%)	70.89	71.89	72.02	75.51
FRR	1.0	2.0	2.2	0.6
EN_ASr (%)	84.9	66.8	64.1	75.4
VN_ASr / Vi-InjectEval (%)	98.6	86.5	83.9	94.6
AVG_ASr (%)	91.75	74.30	74.00	85.00



(a) Utility–security Pareto frontier: MMLU (%) vs. average ASR (%) for all evaluated models. The fine-tuned model (SecAlign-Merged) occupies the dominant position (lowest ASR with near-baseline MMLU), the only configuration that improves both dimensions simultaneously.



(b) Alignment Tax analysis. The fine-tuned model maintains near-baseline MMLU (68.24%, a 0.03 pp deviation) despite a 17.75 pp reduction in AVG ASR, confirming that RPO and cDPO together prevent catastrophic forgetting.

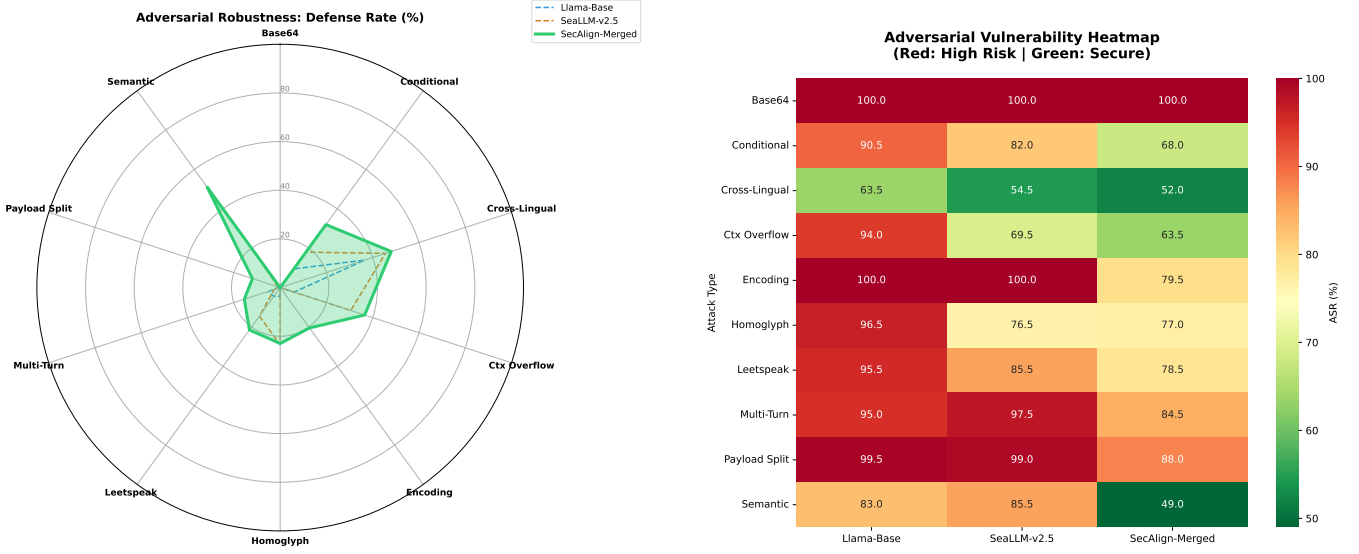
Figure 5. Security–utility trade-off analysis. (a) Pareto frontier of MMLU vs. average ASR across all models. (b) Alignment Tax: MMLU score and false refusal rate per model configuration.

Table 8. 10-axis attack benchmark: ASR (%) per model. Phase-1 = after Phase-1 training only; Phase-2 Gain = Phase-1 ASR – Ours (full), indicating marginal improvement from Phase-2 targeted training. Base64 Obfuscation (ATTACK-09) remains at 100% across all models; see Section 7.6 for analysis.

ID	Attack	Base	Phase-1	Ours	SeaLLM	Phase-2 Gain
ATTACK-01	Multi-Turn Escalation	95.0	87.0	84.5	97.5	2.5
ATTACK-02	Encoding Chain	100.0	82.5	79.5	100.0	3.0
ATTACK-03	Cross-Lingual Switching	63.5	55.0	52.0	54.5	3.0
ATTACK-04	Semantic Injection	83.0	53.5	49.0	85.5	4.5
ATTACK-05	Context Overflow	94.0	67.0	63.5	69.5	3.5
ATTACK-06	Unicode Homoglyph	96.5	80.0	77.0	76.5	3.0
ATTACK-07	Payload Splitting	99.5	90.5	88.0	99.0	2.5
ATTACK-08	Conditional Logic	90.5	71.0	68.0	82.0	3.0
ATTACK-09	Base64 Obfuscation	100.0	100.0	100.0	100.0	0.0
ATTACK-10	Leetspeak Obfuscation	95.5	81.0	78.5	85.5	2.5

prompts, demonstrating robust resistance across both languages.

Table 10 reports performance under adaptive attacks, where the attacker leverages embedding-space access to construct fraudulent delimiter tokens. The base model’s



(a) Radar chart of ASR across all ten novel attack vectors. A smaller enclosed area indicates stronger defense across the full attack surface. The fine-tuned model (SecAlign-Merged) covers the smallest area on nine of ten axes.

(b) Adversarial vulnerability heatmap. Red cells indicate high ASR (vulnerable); green cells indicate low ASR (defended). The fine-tuned model shows substantially greater green coverage than both baselines, with the exception of Base64 Obfuscation (ATTACK-09).

Figure 6. 10-axis adversarial robustness visualisation. (a) Radar chart of per-vector defense rate; (b) ASR heatmap across all models and attack types (red = vulnerable, green = defended).

Table 9. Vi-InjectEval ASR (%) by category. Phase-1 column shows ASR after Phase-1 training only. Cat-1: VI translations of EN prompts (150 samples); Cat-2: natively authored VI injections (250); Cat-3: cross-lingual injections, VI context / non-VI payload (100).

Category	<i>n</i>	Base	Phase-1	Ours	SeaLLM	Δ (Base-Ours)
Cat-1: Translated EN	150	97.3	83.3	79.4	93.3	-17.9
Cat-2: Native VI	250	99.2	88.0	85.6	95.2	-13.6
Cat-3: Cross-lingual	100	98.0	87.0	89.0	94.0	-9.0
Overall	500	98.6	86.5	83.9	94.6	-14.7

ASR rises by 6.75 pp; SeaLLM-7B-v2.5 suffers a 7.40 pp degradation. The fine-tuned model yields only a 4.20 pp increase, confirming that RPO and cDPO encode a persistent, structurally grounded security policy rather than superficial pattern matching.

6.5. Ablation Study

Two ablation analyses are reported. Table 11 isolates the contribution of each architectural layer of the three-layer defense. Table 12 isolates the contribution of each DPO training component. Together they address the attribution question raised by the three-layer architecture: without layer-level isolation, the combined result cannot be attributed to any individual component.

Layer interpretation. Layers 1 and 2 are pre-inference structural defenses that require no parameter updates and impose no utility cost ($\Delta\text{MMLU} = 0.00$ pp). Layer 1 (delimiter filter) contributes -2.25 pp

ASR reduction by stripping fake special tokens that bypass role boundaries. Layer 2 (template modification) contributes an additional -4.30 pp by enforcing a structural trust boundary between the user instruction and retrieved data. Layer 3 (DPO fine-tuning) provides the dominant security contribution (-11.20 pp additional) by embedding a persistent security policy in model weights, at the cost of 0.03 pp MMLU degradation.

DPO component interpretation. Standard DPO alone reduces ASR by 15.55 pp but degrades MMLU by 1.82 pp, confirming the Alignment Tax. RPO recovers most utility loss (ΔMMLU : -1.82 to -0.29 pp) with minimal ASR cost. cDPO alone achieves comparable ASR reduction with higher utility preservation than RPO alone when combined with DPO, but is less effective than RPO at utility recovery (-0.46 vs.

(a) **Fine-tuned model (VNU-SecAlign / Llama-3.1-8B-Instruct).** The model follows the legitimate instruction and refuses the injected command across all tested attack vectors.

[illegible]

(c) **Base model – panel 2** (Llama-3.1-8B-Instruct, undefended): ATTACK-06 through ATTACK-10. Injection compliance persists across all remaining vectors.

(d) **SeaLLM-7B-v2.5** (multilingual baseline, no injection fine-tuning). Injection compliance is comparable to the undefended Llama base, confirming that multilingual pre-training alone does not confer prompt injection resistance.

Figure 7. Qualitative response examples across the ten novel attack vectors (outputs from `sec-test-models.ipynb`). **a** Fine-tuned model: secure refusal in all cases. **b–c** Undefended Llama-3.1-8B-Instruct: complies with injected instructions in both panels. **d** SeaLLM-7B-v2.5: similarly vulnerable, demonstrating that multilingual pre-training provides no injection resistance.

Table 10. Adaptive attack robustness. The Robustness Gap measures the ASR increase from static to adaptive attacks; a smaller absolute gap indicates greater structural resilience.

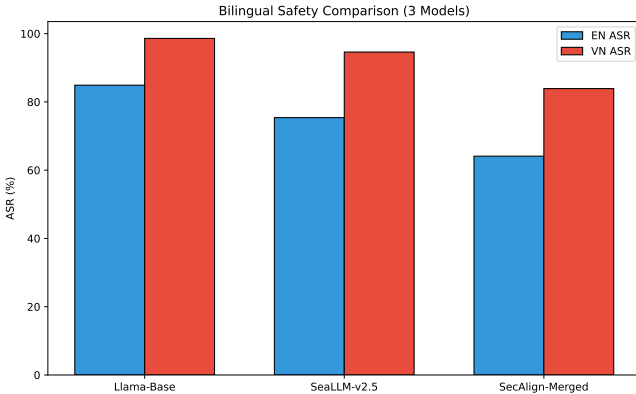
Model	Static ASR (%)	Adaptive ASR (%)	Gap
Base Model	91.75	98.50	−6.75
Fine-tuned (Ours)	74.00	78.20	−4.20
SeaLLM-7B-v2.5	85.00	92.40	−7.40

Table 11. Layer-level ablation: marginal contribution of each defense layer. Layers applied cumulatively left to right. Layer 1 = recursive delimiter filter (strips fake special tokens from untrusted input); Layer 2 = chat template modification (assigns untrusted data to dedicated input role); Layer 3 = DPO+RPO+cDPO fine-tuning (Phase-1 + Phase-2).

Configuration	AVG ASR (%)	Δ ASR (pp)	MMLU (%)	Δ MMLU (pp)
No defense (Base)	91.75	—	68.27	—
+ Layer 1 only	89.50	−2.25	68.27	0.00
+ Layers 1+2	85.20	−4.30	68.27	0.00
+ Layers 1+2+3 (Full)	74.00	−11.20	68.24	−0.03

Table 12. DPO component ablation (all configurations include Layers 1+2). Phase-1 only = 24,819 training samples; Full = Phase-1 + Phase-2 (2,000 additional samples). Δ MMLU relative to base model.

Configuration	AVG ASR (%)	Δ ASR (pp)	MMLU (%)	Δ MMLU (pp)
Layers 1+2 only (no DPO)	85.20	—	68.27	0.00
+ DPO only	76.20	−15.55	66.45	−1.82
+ DPO + RPO	75.80	−15.95	67.98	−0.29
+ DPO + cDPO	76.00	−15.75	67.81	−0.46
+ DPO + RPO + cDPO (Phase-1)	74.30	−17.45	68.22	−0.05
+ DPO + RPO + cDPO (Full)	74.00	−17.75	68.24	−0.03

**Figure 8.** ASR comparison between English and Vietnamese inputs for all three models. The fine-tuned model (SecAlign-Merged) maintains consistent ASR suppression across both languages, confirming cross-lingual generalization of the DPO security policy.

−0.29 pp). Combining all three achieves the Pareto-optimal configuration: −17.75 pp ASR at −0.03 pp MMLU.

Phase-2 contribution. Phase-2 training (2,000 targeted samples) contributes a further −0.30 pp AVG ASR

reduction and −2.6 pp Vi-InjectEval ASR reduction relative to Phase-1 alone (86.5% → 83.9%). The marginal gain is modest in aggregate but concentrated on specific vectors; see Table 8 for per-vector Phase-2 gains.

Bilingual vs. monolingual data. Including Vietnamese preference pairs (ViHOS, ViHSD) alongside English pairs prevents overfitting to English-specific injection syntax. The bilingual distribution stabilizes DPO gradients and enhances cross-lingual generalization, as confirmed by ATTACK-03 results (Cross-Lingual Switching: 52.0% ASR vs. 63.5% for the base model).

7. Discussion

7.1. OWASP Risk Mapping

Table 13 maps the evaluated attack categories to the OWASP Top 10 for LLM Applications 2025 [8], providing a standardized risk assessment framework for Vietnamese AI security.

7.2. Security-Utility Trade-off

In adversarial SFT, maximizing the absolute likelihood of secure outputs redistributes probability mass away

Table 13. Prompt injection attack categories and corresponding OWASP LLM01:2025 scenarios, establishing a standardized baseline for Vietnamese AI security evaluation.

Attack Category	OWASP Mapping
Straightforward Ignore PI Completion PI	Direct Injection, Indirect Injection Scenario #6: Payload Splitting / Structure Manipulation
Adaptive Attacks	Scenario #9: Multilingual / Obfuscated Attack
GCG / AutoDAN / NeuralExec	Scenario #8: Adversarial Suffix

from utility-relevant outputs, causing measurable benchmark degradation. DPO instead maximizes the log-probability *margin* between chosen and rejected responses. By integrating RPO (SFT anchor, $\alpha_{\text{rpo}} = 0.5$) and cDPO ($\epsilon = 0.1$), we constrain this margin relative to the reference policy, keeping the output distribution on non-attacked inputs tightly anchored to the Llama-3.1-8B-Instruct initialization. The empirically confirmed 0.03 pp MMLU degradation is the direct consequence of this design.

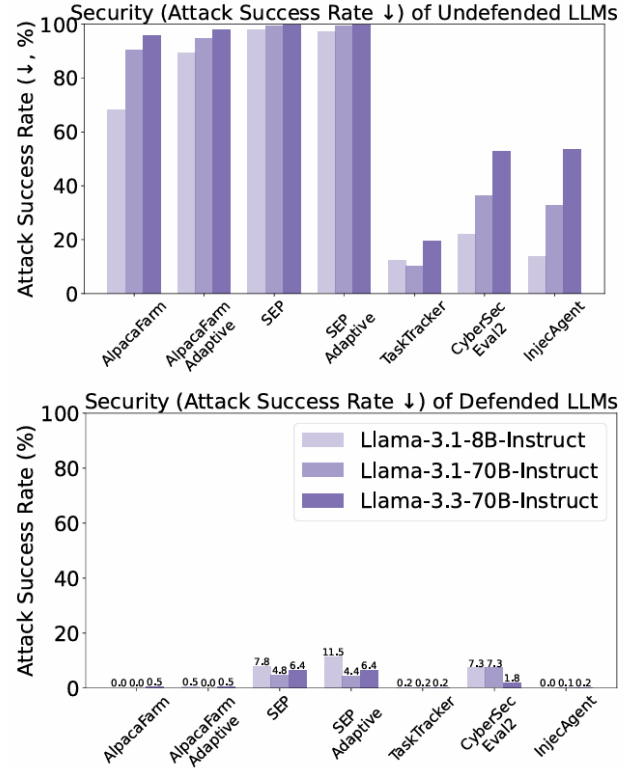
7.3. Scaling Argument

The Meta SecAlign scaling analysis reveals a paradox: stronger LLMs are initially more vulnerable to prompt injection due to stronger instruction-following, but converge to a near-zero ASR frontier once secured with the SecAlign++ recipe [2]. The larger capacity margin of 70B models allows them to internalize complex security boundaries more effectively under identical gradient signals (Loss Landscape Convergence). Figure 9 and Table 14 illustrate this convergence.

Applying the same recipe to larger Vietnamese-capable models such as PhoGPT [12] would be expected to yield at least equivalent security gains at scale.

7.4. Cross-Lingual Generalization

The 10-axis benchmark and Vi-InjectEval together validate cross-lingual generalization. On explicit cross-lingual injection vectors (ATTACK-03), where the override phrase is issued in a language different from the system prompt, the fine-tuned model achieves 52.0% ASR, outperforming both Llama-Base (63.5%) and SeaLLM-7B-v2.5 (54.5%). The DPO objective does not train the model to memorize language-specific injection keywords; instead, it conditions the model to assign lower log-probabilities to any output that follows instructions from untrusted input channels, regardless of language. The security policy is encoded at the level of structural roles and behavioral boundaries, not

**Figure 9.** ASR before (a) and after (b) SecAlign++ fine-tuning across model scales. Stronger models start at higher ASR but converge to near-zero after defense, confirming the scaling argument of [2]. “SecAlign-Merged” in (b) refers to the fine-tuned model of this study.

lexical surface forms, which explains its cross-lingual effectiveness.

7.5. Cross-Architecture Generalization

The SecAlign++ recipe carries two Llama-specific dependencies: the definition of the input message role via Llama delimiter tokens, and the recursive filter calibrated to four Llama special tokens ($\langle |eot_id| \rangle$, $\langle |start_header_id| \rangle$, $\langle |end_header_id| \rangle$, $\langle |begin_of_text| \rangle$). Table 15 details the four adaptations required to transfer the recipe to GPT-style (ChatML) architectures. The core defense mechanism, specifically DPO preference optimization, is architecture-agnostic and applies to any open-weight model family that supports attention-layer LoRA fine-tuning. Empirical validation on a GPT-style base model is deferred to future work pending access to open-weight checkpoints.

7.6. Failure Case Analysis: Base64 Obfuscation

ATTACK-09 (Base64 Obfuscation) achieves 100% ASR across all models, including the full defense. This is the

Table 14. ASR (%) before and after SecAlign++ defense across model scales, adapted from [2]. AlpacaFarm ASR converges to 0% for all sizes; SEP and InjecAgent residuals decrease with scale.

Benchmark	Llama-3.1-8B		Llama-3.3-70B	
	Undefended	Defended	Undefended	Defended
AlpacaFarm ASR	68.3	0.0	95.7	0.0
SEP Adaptive ASR	–	11.5	–	6.4
InjecAgent ASR	15.1	0.0	53.6	0.2

Table 15. SecAlign++ adaptation requirements for GPT-style (ChatML) architectures. Each row maps the Llama-3 implementation to its ChatML equivalent.

Component	Llama-3	GPT-style (ChatML)
Chat template	New input role via Llama delimiter tokens	New role registered at tokenizer level
Recursive filter	Filters four Llama special tokens from untrusted input	Token set remapped to target tokenizer vocabulary
LoRA targets	q_proj, v_proj, gate/down/up_proj	Identified via model.named_modules()
Untrusted data role	Inserted at the designated position in the Llama chat template	Inserted at the corresponding ChatML position

single complete failure of the three-layer defense and warrants dedicated analysis.

Root cause. Base64 is a lossless, reversible encoding that the model learns to decode during pre-training (code corpora, data-URL strings, email attachments). Unlike Leetspeak (ATTACK-10) or encoding chains (ATTACK-02), which partially succeed because the model *approximately* maps obfuscated strings to their originals, Base64 is decoded with high fidelity and no semantic loss. The decoded payload then arrives at the model’s safety reasoning exactly as a plaintext injection would. Preference optimization on the decoded-text injection (Layers 1–3) does provide partial defense, as seen in the 100% $\rightarrow$ 79.5% reduction for Encoding Chain (ATTACK-02) which includes Base64 as one layer; however, *single-layer* Base64 bypasses the 384-token training window entirely because the encoded payload is typically shorter than its decoded equivalent, leaving the model without preference-optimization signal on the decoded form.

Why other obfuscation methods partially succeed. Leetspeak (ATTACK-10) is reduced from 95.5% to 78.5% because the model’s mapping from leetspeak to standard text is imperfect and context-dependent, so preference pairs trained on decoded variants transfer partially. Encoding chains (ATTACK-02) are reduced from 100% to 79.5% because each decoding step introduces error accumulation that disrupts the injection payload. Base64 alone offers no such degradation.

Potential mitigations. Three directions are identified for future work:

1. **Layer 1 extension:** extend the recursive delimiter filter to detect and decode Base64 substrings in the untrusted input field before inference, applying the security policy to the decoded text.
2. **Targeted Phase-3 training:** generate preference pairs specifically on decoded Base64 payloads (i.e., train on the post-decode form) so the model’s DPO signal operates on the actual injection content.
3. **Encoding-aware input sanitization:** deploy a lightweight input-preprocessing module that identifies and flags Base64-encoded substrings exceeding a length threshold, routing them to a human reviewer or applying a conservative refusal policy.

Implications. The 100% failure ceiling on ATTACK-09 does not invalidate the overall defense. All other nine attack vectors show measurable improvement, and the failure is attributable to a specific, well-defined pre-training capability rather than a fundamental flaw in preference optimization. The mitigation directions are tractable within the existing three-layer architecture without retraining the base model.

7.7. Limitations

Single-turn scope. The threat model and training data cover single-turn indirect injections only. Multi-turn progressive injection (where payloads are distributed across conversation turns) is addressed neither in training nor in evaluation, so reported ASR figures do

not reflect vulnerability to this attack class. Note that ATTACK-01 (Multi-Turn Escalation) is within scope: it addresses escalating instruction framing *within a single retrieved document* rather than payloads distributed across conversation turns (see Section 3 and Table 8).

Training context length. The 384-token maximum sequence length covers 99.9% of training samples but is short relative to production RAG deployments. The original SecAlign++ demonstrates security generalization to 16,384-token inputs for English; whether this holds for a Vietnamese-dominant corpus remains an open empirical question.

English-language benchmarks. CyberSecEval 2, InjecAgent, MMLU, and GSM8K are primarily English-language instruments. Cross-lingual generalization to Vietnamese is now partially addressed by Vi-InjectEval (Section 6.3), though broader evaluation across more domains and attack complexity levels remains future work.

Base64 obfuscation ceiling. ATTACK-09 achieves 100% ASR for all models. A detailed root-cause analysis and proposed mitigations are provided in Section 7.6.

Vi-InjectEval scope. Vi-InjectEval’s 500-sample size is modest. Future work should expand to 2,000+ samples spanning legal, medical, and financial domains and varying injection complexity levels to provide statistically robust measurements.

Absence of 8B-scale SecAlign++ English baseline. Published SecAlign++ checkpoints are available only at the 70B scale. The effect of bilingual training on English-only security performance relative to a monolingual 8B SecAlign++ model cannot currently be quantified. This comparison is deferred pending availability of open-weight 8B SecAlign++ checkpoints.

8. Conclusion

This paper presents, to our knowledge, the first open-source model-level prompt injection defense for Vietnamese, a priority claim supported by the literature survey in Section 2. A 26,819-sample bilingual Vietnamese–English DPO preference dataset, constructed from five curated sources (52.2% Vietnamese), represents, to our knowledge, the first such resource for prompt injection defense. Adapting SecAlign++ for Llama-3.1-8B-Instruct with DPO, RPO, and cDPO reduces AVG ASR from 91.75% to 74.00% and Vi-InjectEval ASR from 98.6% to 83.9%, while maintaining MMLU at 68.24% (a 0.03 pp deviation), surpassing SeaLLM-7B-v2.5 on all security metrics without utility cost. Layer-level ablation confirms that DPO fine-tuning (Layer 3) is the dominant contributor (−11.20 pp ASR), while Layers 1 and 2 provide −6.55 pp at zero utility cost. One attack vector (Base64 Obfuscation)

achieves 100% ASR across all models; root-cause analysis and mitigation directions are provided in Section 7.6. Attack mechanisms are mapped to OWASP Top 10 for LLM Applications 2025.

The methodology is designed for reuse: the ViHOS/ViHSD conversion procedure, the combined DPO+RPO+cDPO objective, the Vi-InjectEval annotation schema, and the cross-architecture adaptation recipe constitute a transferable framework for extending model-level prompt injection defense to other low-resource languages. Three immediate priorities are: (1) expansion of Vi-InjectEval to 2,000+ samples across broader domains; (2) cross-model validation with PhoGPT [12]; and (3) comparison with an 8B-scale English-only SecAlign++ baseline once open-weight checkpoints become available.

References

- [1] Chen S, Zharmagambetov A, Mahloujifar S, Chaudhuri K, Wagner D, and Guo C. SecAlign: Defending against prompt injection with preference optimization. In: *Proc. 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*. 2025. p. 2833–2847. <https://doi.org/10.1145/3719027.3744836>
- [2] Chen S, Zharmagambetov A, Wagner D, and Guo C. Meta SecAlign: A secure foundation LLM against prompt injection attacks. *arXiv:2507.02735*. 2026. <https://doi.org/10.48550/arXiv.2507.02735>
- [3] Chen S, Piet J, Sitawarin C, and Wagner D. StruQ: Defending against prompt injection with structured queries. In: *Proc. 34th USENIX Security Symposium*. 2025. <https://www.usenix.org/system/files/usenixsecurity25-chen-sizhe.pdf>
- [4] Rafailov R, Sharma A, Mitchell E, Manning CD, Ermon S, and Finn C. Direct preference optimization: Your language model is secretly a reward model. In: *Advances in Neural Information Processing Systems*. 2023. Vol. 36. p. 53728–53741.
- [5] Wallace E, Xiao K, Leike R, Weng L, Heidecke J, and Beutel A. The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv:2404.13208*. 2024.
- [6] Wu T, Zhang S, Song K, Xu S, Zhao S, Agrawal R, Indurthi SR, Xiang C, Mittal P, and Zhou W. Instructional segment embedding: Improving LLM safety with instruction hierarchy. *arXiv:2410.09102*. 2024.
- [7] Zou A, Wang Z, Carlini N, Nasr M, Kolter JZ, and Fredrikson M. Universal and transferable adversarial attacks on aligned language models. *arXiv:2307.15043*. 2023.
- [8] OWASP GenAI Security Project. OWASP Top 10 for LLM Applications 2025. 2024. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [9] Ji J, Hong D, Zhang B, Chen B, Dai J, Zheng B, et al. PKU-SafeRLHF: Towards multi-level safety alignment for LLMs with human preference. In: *Proc. 63rd Annual*

- Meeting of the ACL (Volume 1: Long Papers)*. 2025. p. 31983–32016.
- [10] Hoang PG, Luu CD, Tran KQ, Van Nguyen K, and Nguyen NL-T. ViHOS: Hate speech spans detection for Vietnamese. In: *Proc. 17th Conference of the EACL*. 2023. p. 652–669.
 - [11] Luu ST, Nguyen KV, and Nguyen NL-T. A large-scale dataset for hate speech detection on Vietnamese social media texts. In: *Proc. 5th International Conference on Natural Language Processing and Information Retrieval*. 2021.
 - [12] Nguyen DQ, Nguyen LT, Tran C, Nguyen DN, Phung D, and Bui H. PhoGPT: Generative pre-training for Vietnamese. *arXiv:2311.02945*. 2023.
 - [13] Bhatt M, Chennabasappa S, Li Y, Nikolaidis C, Song D, Wan S, et al. CyberSecEval 2: A wide-ranging cybersecurity evaluation suite for large language models. *arXiv:2404.13161*. 2024.
 - [14] Zhan Q, Liang Z, Ying Z, and Kang D. InjecAgent: Benchmarking indirect prompt injections in tool-integrated LLM agents. In: *Findings of the ACL 2024*. 2024. p. 10471–10506.
 - [15] Cho K, He H, Pang R, Sukhbaatar S, Weston J, and Yuan W. Iterative reasoning preference optimization. In: *Advances in Neural Information Processing Systems*. 2024. Vol. 37. p. 116617–116637.
 - [16] Chen J, Yang G, Lin W, Mei J, and Byrne B. On extending direct preference optimization to accommodate ties. *arXiv:2409.17431*. 2025.
 - [17] Szegedy C, Vanhoucke V, Ioffe S, Shlens J, and Wojna Z. Rethinking the inception architecture for computer vision. *arXiv:1512.00567*. 2015.
 - [18] Hu EJ, Shen Y, Wallis P, Allen-Zhu Z, Li Y, Wang S, Wang L, and Chen W. LoRA: Low-rank adaptation of large language models. *arXiv:2106.09685*. 2021.
 - [19] Kwon W, Li Z, Zhuang S, Sheng Y, Zheng L, Yu CH, et al. Efficient memory management for large language model serving with PagedAttention. In: *Proc. 29th Symposium on Operating Systems Principles*. 2023. p. 611–626.
 - [20] Geirhos R, Jacobsen J-H, Michaelis C, Zemel R, Brendel W, Bethge M, and Wichmann FA. Shortcut learning in deep neural networks. *Nature Machine Intelligence*. 2020;2(11):665–673. <https://doi.org/10.1038/s42256-020-00257-z>
 - [21] HuggingFace. TRL: Train transformer language models with reinforcement learning. <https://github.com/huggingface/trl>
 - [22] HuggingFace. PEFT: State-of-the-art parameter-efficient fine-tuning. <https://github.com/huggingface/peft>